

Slide 1



Helm

Complete Roadmap

Kubernetes Package Manager



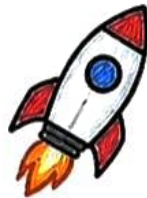
Charts

- Reusable K8s packages



Templates

- Dynamic YAML generation



Deployments

- One command to deploy everything



Rollbacks

- Easy rollbacks & version control



Production Ready

- Consistent, Reliable & Scalable

Everything you need to
master Helm!

• @NeerajSingh-DevOps

Slide 2

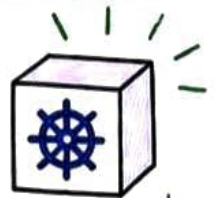
What is Helm?



Helm is the **package manager** for Kubernetes.

It helps you **define, install, and manage** Kubernetes applications using **Charts**.

Instead of applying multiple YAML files manually, Helm deploys everything with a **single command**.



Why Helm?

- ✓ **Faster Deployments**
Install complex applications in seconds.



- ✓ **Reusable Templates**
Create once and use again and again.



- ✓ **Easy Upgrades**
Upgrade applications with zero downtime.



- ✓ **Quick Rollbacks**
Roll back to the previous version easily.



Example

```
# Install a chart  
helm install myapp ./mychart
```

```
# Upgrade  
helm upgrade myapp ./mychart
```

```
# Rollback  
helm rollback myapp 1
```

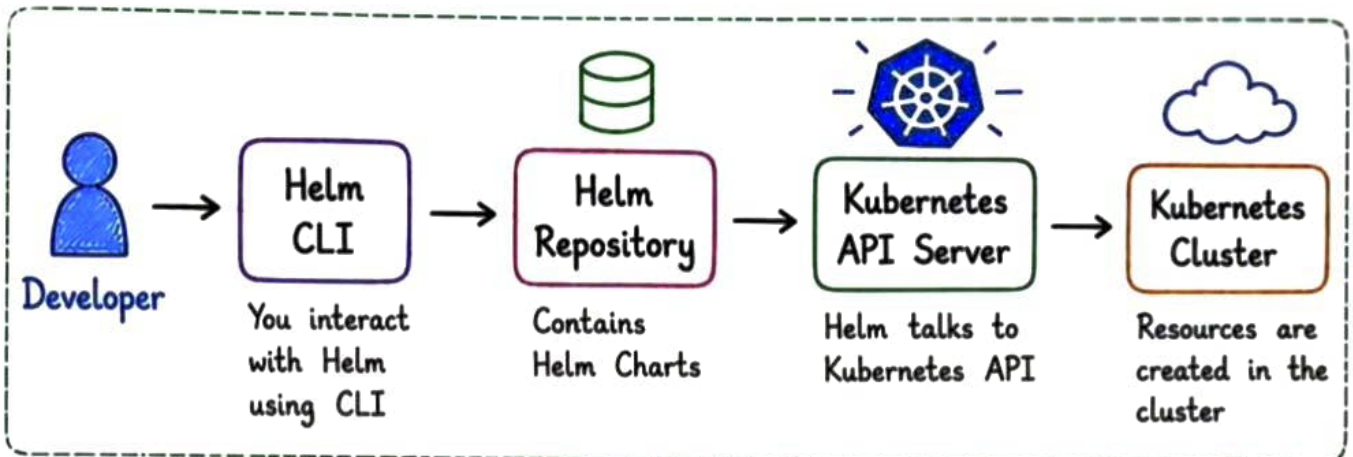
```
# Uninstall  
helm uninstall myapp
```

One command,
everything deployed!

★ Helm makes Kubernetes deployments **Simple, Consistent & Powerful**. :o:

Slide 3

Helm Architecture



How Helm Works ?

- You create or use a Helm Chart.
- Helm packages the Kubernetes manifests.
- Helm sends the manifests to Kubernetes API Server.
- Kubernetes deploys the resources in the cluster.
- Helm tracks the release and manages changes.



Key Components

- ★ **Helm CLI** : Command line tool to manage charts and releases.
- ★ **Helm Repository** : Store charts (public or private).
- ★ **Helm Chart** : Collection of files that describe related Kubernetes resources.
- ★ **Kubernetes API Server** : Validates and processes the request.
- ★ **Kubernetes Cluster** : Runs and manages the actual resources.

Benefits of Helm Architecture



Consistent Deployments



Easy Upgrades



Quick Rollbacks



Reusability of Charts



Team Collaboration

★ Helm = **Simpler, Faster and Smarter** Kubernetes Deployments!

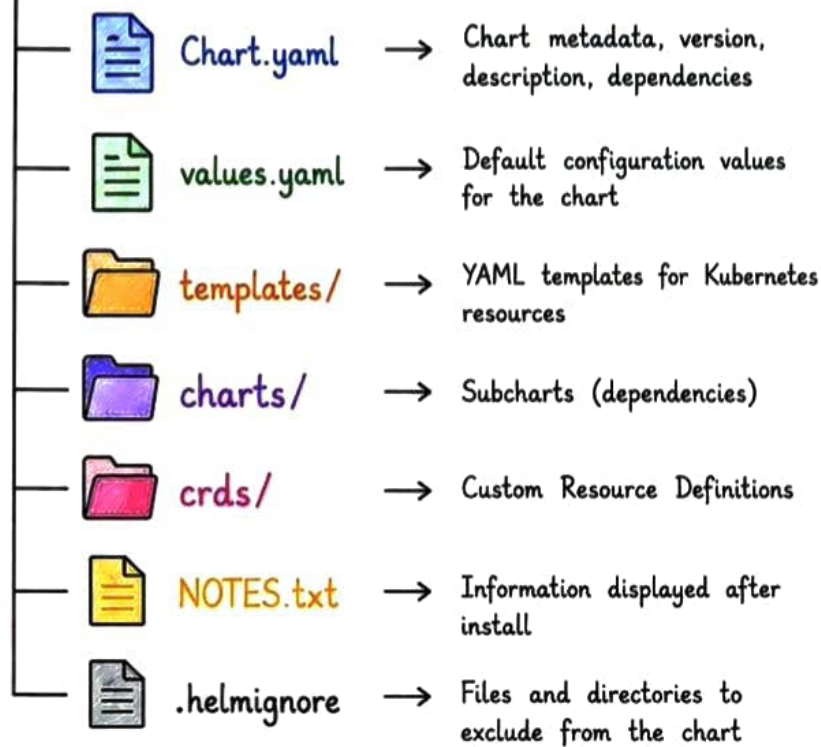
Slide 4

Helm Chart Structure

A **Helm Chart** is a collection of files that describe a related set of **Kubernetes** resources. Below is the default structure of a Helm Chart.



mychart/



What does each folder do?

- **Chart.yaml**
Contains information about the chart.
- **values.yaml**
Holds configurable values. These can be overridden during install/upgrade.
- **templates/**
Contains the Kubernetes manifest templates.
- **charts/**
Used to manage dependent charts.
- **crds/**
Stores CRD YAML files.
- **NOTES.txt**
Helpful notes shown after release is installed.
- **.helmignore**
Works like **.gitignore** to ignore files while packaging.

Why is this structure important?

- ✓ Keeps everything organized
- ✓ Makes charts reusable and shareable
- ✓ Follows Helm best practices
- ✓ Easy to version control and maintain



Helm Charts = **Reusable**, **Shareable** and **Maintainable** Kubernetes Packages **Smart**

Slide 5

Chart.yaml vs values.yaml

≡ Two Important Files in Every Helm Chart ≡



Chart.yaml

Contains **metadata** about the chart.

Key Fields

- **apiVersion** → Helm API version
- **name** → Name of the chart
- **version** → Chart version
- **description** → Short description
- **type** → Application / Library
- **appVersion** → Version of the application
- **dependencies** → List of chart dependencies
- **maintainers** → Author / Maintainer info

Example:

```
apiVersion: v2
name: myapp
description: A sample application
type: application
version: 1.0.0
appVersion: "1.0"
maintainers:
  - name: Sunke Nagarani
    email: rani@example.com
```



values.yaml

Contains default configuration values for the chart.

Key Sections

- **image** → Container image details
- **replicaCount** → Number of replicas
- **service** → Service type and port
- **resources** → CPU/Memory limits
- **env** → Environment variables
- **ingress** → Ingress configuration
- **persistence** → Storage configuration
- **other custom** → Any custom values

Example:

```
image:
  repository: nginx
  tag: "1.25"
  pullPolicy: IfNotPresent
replicaCount: 2
service:
  type: ClusterIP
  port: 80
resources:
  limits:
    cpu: "500m"
    memory: "512Mi"
  requests:
    cpu: "200m"
    memory: "256Mi"
```



In Simple Words

Chart.yaml tells Helm "What is this chart?"

values.yaml tells Helm "How to configure it?"



Understand these two files well,

You understand **50%** of Helm!



Slide 6

Helm Templates

Helm uses the **Go Template** engine to generate Kubernetes manifests dynamically.



Common Template Features

`{{.Values.name}}`

Variables

- Store values and reuse them.



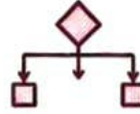
Functions

- Built-in functions simplify tasks.



Pipelines

- Pass the output of one command to another.



Conditionals

- Render content based on conditions.



Loops

- Repeat blocks of content multiple times.



Includes

- Reuse parts of templates using include.

Example Template (Deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Values.name }}
  labels:
    app: {{ .Values.name }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Values.name }}
  template:
    metadata:
      labels:
        app: {{ .Values.name }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
          ports:
            - containerPort: {{ .Values.service.port }}
```

Example values.yaml

```
name: myapp
replicaCount: 2
image:
  repository: nginx
  tag: "1.25"
service:
  type: ClusterIP
  port: 80
resources:
  limits:
    cpu: "500m"
    memory: "512Mi"
ingress:
  enabled: true
  host: myapp.example.com
```



Templates make charts flexible, reusable and powerful!

Pro Tip



- ☒ Use meaningful variable names.
- ☒ Use include and define to avoid repetition.
- ☒ Keep templates simple and readable.
- ☒ Always test templates using "helm template".



Good templates = Clean deployments + Happy DevOps! 😊

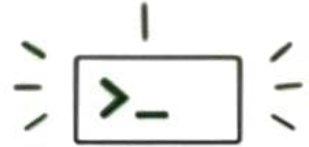


Slide 7

Helm Commands



Essential Helm CLI commands every DevOps engineer **must** know!



Command	Description
 <code>helm version</code>	Check Helm client and server versions.
 <code>helm repo add <name> <url></code>	Add a new Helm repository.
 <code>helm repo update</code>	Update information of all repositories.
 <code>helm search repo <keyword></code>	Search charts in repositories.
 <code>helm create <chart-name></code>	Create a new Helm chart.
 <code>helm install <release> <chart></code>	Install a chart as a release.
 <code>helm upgrade <release> <chart></code>	Upgrade an existing release.
 <code>helm rollback <release> <revision></code>	Rollback to a previous revision.
 <code>helm uninstall <release></code>	Uninstall a release.
 <code>helm list</code>	List all releases.
 <code>helm status <release></code>	Show status of a release.
 <code>helm history <release></code>	Show revision history of a release.

Common Workflow

- ① Add Repo
- ↓
- ② Search Chart
- ↓
- ③ Install
- ↓
- ④ Upgrade
- ↓
- ⑤ Rollback (if needed)
- ↓
- ⑥ Uninstall

Tip



- ★ Always update repos before installing.
- ★ Use meaningful release names.
- ★ Check status and history before rollback.
- ★ Use `--dry-run` to test before install/upgrade.

Useful Extra Commands

- `helm lint <chart>` → Check chart for potential issues.
- `helm template <chart>` → Render templates locally (debugging).
- `helm pull <chart>` → Download a chart.
- `helm show <chart>` → Show information about a chart.



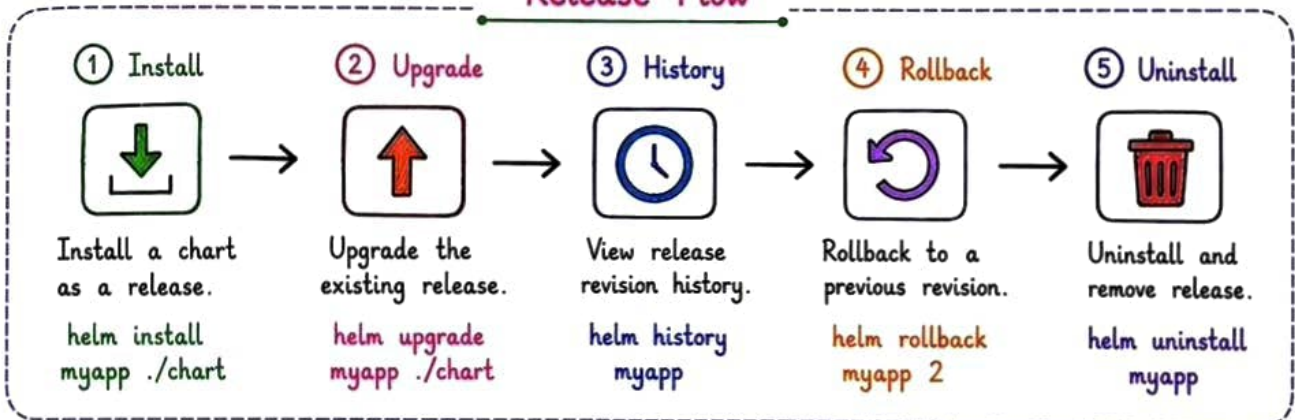
Learn these commands. Practice daily. Become a Helm Expert!

Slide 8

Helm Release Lifecycle

A Helm **Release** is an instance of a chart running in a **Kubernetes** cluster.

Release Flow



What Happens in Each Step?

- 1 Install** : Helm renders templates and creates Kubernetes resources.
- 2 Upgrade** : Helm computes the difference and updates the resources.
- 3 History** : Helm stores every revision of the release.
- 4 Rollback** : Helm rolls back to the specified revision.
- 5 Uninstall** : Helm deletes all resources of the release.

Benefits of Release Management

- ✓ Version controlled deployments
- ✓ Easy upgrades and rollbacks
- ✓ Track changes and history
- ✓ Minimize downtime
- ✓ Recover from failures quickly
- ✓ Consistent and repeatable deployments



Helm keeps your releases **safe**, **trackable** and **easy to manage**!

Quick Tip

- ★ Always use meaningful release names.

Example: `myapp-prod`,
`myapp-dev`

Release States

- **DEPLOYED** : Release is successfully deployed.
- **FAILED** : Release failed during install/upgrade.
- **PENDING** : Release is in progress.
- **SUPERSEDED** : Newer release has replaced this one.
- **DELETED** : Release has been uninstalled.

Remember

Every upgrade creates a new revision!



Use Helm Release Lifecycle to **Deploy**, **Upgrade**, **Rollback** and **Succeed**!

Slide 9

Helm Repositories

- Helm charts are stored in **repositories**.
- You can use **public**, **private** or **OCI** based repositories.

Types of Helm Repositories

① Public Repositories



- Open to everyone.
- No authentication required.
- Examples:
 - Bitnami
 - Artifact Hub
 - Helm Hub

② Private Repositories



- Restricted access.
- Requires authentication.
- Useful for company internal charts.
- Examples:
 - JFrog Artifactory
 - Nexus Repository
 - Harbor

③ OCI Based Repositories



- Store charts as OCI artifacts.
- Use Docker registry protocol.
- Examples:
 - Docker Hub
 - GitHub Container Registry
 - AWS ECR

④ Local Repositories



- Stored locally on your machine.
- Useful for testing and development.
- No network dependency.

Popular Public Repositories

- Bitnami
URL: <https://charts.bitnami.com/bitnami>
- Artifact Hub
URL: <https://artifacthub.io/>
- Helm Hub
URL: <https://hub.helm.sh/>



Why Use Repositories?

- ★ Share charts easily
- ★ Reuse charts
- ★ Keep charts versioned
- ★ Standard way to distribute charts



Common Helm Repository Commands

- | | |
|---|---|
|  <code>helm repo add <name> <url></code> | → Add a new repository. |
|  <code>helm repo update</code> | → Update information of all repositories. |
|  <code>helm repo list</code> | → List all configured repositories. |
|  <code>helm search repo <keyword></code> | → Search charts in repositories. |
|  <code>helm repo remove <name></code> | → Remove a repository. |

Keep your repositories updated to get the latest charts!



Tip: Always update repositories before searching or installing chart.

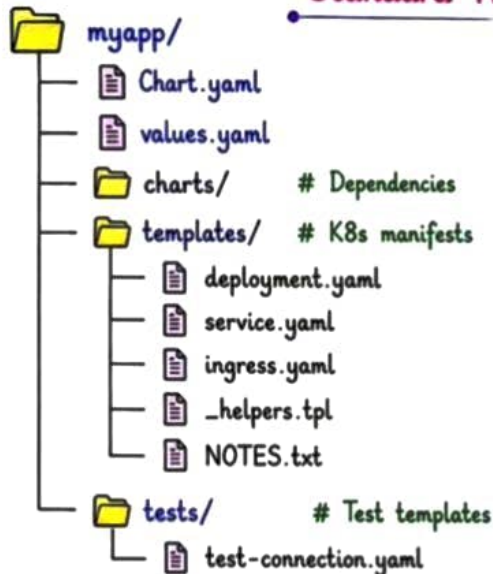
Slide 10

Helm Chart Structure

A Helm chart is a collection of files that describe a related set of Kubernetes resources.



Standard Helm Chart Directory Structure



Key Files Explained	
Chart.yaml	Metadata about the chart (name, version, description, dependencies).
values.yaml	Default configuration values for the chart.
charts/	Stores dependent charts (subcharts).
templates/	Contains Kubernetes manifest templates.
_helpers.tpl	Reusable template snippets using define and include.
NOTES.txt	Displays useful messages after install/upgrade.
tests/	Contains test templates to validate the release.

Example: Chart.yaml (Snippet)

```

apiVersion: v2
name: myapp
description: A sample Helm chart for myapp
type: application
version: 1.0.0
appVersion: "1.0.0"
dependencies:
  - name: nginx
    version: 15.0.0
    repository: https://charts.bitnami.com/bitnami
  
```



Example: values.yaml (Snippet)

```

replicaCount: 2
image:
  repository: nginx
  tag: "1.25"
service:
  type: ClusterIP
  port: 80
ingress:
  enabled: true
  host: myapp.example.com
  
```



Quick Tips



Keep values.yaml clean and well documented.



Use _helpers.tpl for reusable template code.



Keep your chart structure simple and organized.



Well structured chart = Easy to Maintain + Easy to Scale!



Slide 11

Helm Best Practices



Following best practices makes your Helm charts **maintainable**, **secure** and **scalable**.



Best Practices Checklist

- 1 Keep charts **simple** and focused.
- 2 Use **meaningful** and **consistent** names.
- 3 Follow the **standard directory structure**.
- 4 Use **values.yaml** for configurable values.
- 5 Provide sensible **default values**.
- 6 Use **helpers.tpl** for reusable templates.
- 7 **Validate** input values in templates.
- 8 **Document** everything clearly.
- 9 **Test** your charts before publishing.
- 10 **Version** your charts and dependencies.



Keep it simple



Use good naming



Follow structure



Use values.yaml



Set good defaults



Reuse with helpers



Validate inputs



Write clear docs



Test before release



Version everything

Common Pitfalls to Avoid

- ✗ Hardcoding values in templates
- ✗ Big charts doing too many things
- ✗ Ignoring template linting and validation
- ✗ Not testing upgrades and rollbacks
- ✗ Poor documentation
- ✗ Not pinning chart dependencies



Useful Tools

- `helm lint <chart>` : Check chart for issues
- `helm template <chart>` : Render templates locally
- `helm install <chart>` : Install a release
- `helm upgrade <release> <chart>` : Upgrade a release
- `helm rollback <release> <rev>` : Rollback to a revision
- `helm test <release>` : Run tests for a release
- `helm uninstall <release>` : Uninstall a release

Remember

- ★ A good Helm chart is **easy to install**, **easy to configure** and **easy to maintain**!
- ★ Write charts for your future self and for others!



Slide 12

Helm Summary Cheat Sheet



Helm makes Kubernetes application management **simple**, **predictable** and **powerful**.



Helm in One Line

Helm is the **package manager** for Kubernetes.
It helps you package, deploy and manage applications easily.

Most Used Helm Commands

Command	Purpose
 <code>helm version</code>	Check Helm client & server versions
 <code>helm repo add <name> <url></code>	Add a new Helm repository
 <code>helm repo update</code>	Update information of repositories
 <code>helm search repo <keyword></code>	Search charts in repositories
 <code>helm install <release> <chart></code>	Install a chart as a release
 <code>helm upgrade <release> <chart></code>	Upgrade an existing release
 <code>helm rollback <release> <rev></code>	Rollback to a previous revision
 <code>helm uninstall <release></code>	Uninstall a release
 <code>helm list</code>	List all releases
 <code>helm status <release></code>	Show status of a release
 <code>helm history <release></code>	Show revision history

Helm Release Lifecycle (Recap)



Key Takeaways

- ★ Helm packages Kubernetes resources into **charts**.
- ★ Charts promote **reusability** and **consistency**.
- ★ **Repositories** help you discover and share charts.
- ★ **Releases** help you manage application versions.
- ★ Use **upgrade** and **rollback** to manage changes safely.
- ★ Good **documentation** and **structure** save a lot of time.



Pro Tips

- Always update repos before searching. 
- Use **--dry-run** to test before install/upgrade. 
- Write clean `values.yaml` and document it. 
- Keep your charts versioned and tested. 

Helm Chart Files (Quick View)

 <code>Chart.yaml</code>	- Metadata about the chart
 <code>values.yaml</code>	- Default configuration values
 <code>templates/</code>	- Kubernetes manifest templates
 <code>_helpers.tpl</code>	- Reusable template snippets
 <code>NOTES.txt</code>	- Info shown after install/upgrade
 <code>charts/</code>	- Subcharts (dependencies)

Helm Success Formula

Good Structure + Clean Values + Useful Helpers + Testing + Documentation = Success 



Master Helm. Automate Deployments. Deliver Faster. Stay Awesome. Stay